

Operating Systems are crucial in a computer's success. They handle tasks and act as the middleman between user and hardware. Now, handling tasks correctly might not be enough as technology evolves, and computers are expected to be faster and more reliable. Operating Systems not only need to handle processes and communication correctly, but it is expected to do it in the fastest way possible. When computers were first a thing, running one program at a time was enough and highly praised. Fast forward to the present day, this is nowhere near to being acceptable. Users expect to be able to run multiple applications at once and never have to deal with a frozen computer that can't handle the load. This is where threading becomes very crucial in Operating Systems today. Threading allows multiple tasks to run simultaneously, improving program efficiency. It's good to be able to buy your favorite product at a local supermarket, but it's better when you're able to reach the register faster to pay for it and be on your way. One register open can check everybody out, but multiple registers can check everybody out faster.

Understanding threads can be quite simple once you understand processes, as a thread is simply a unit of execution within a process. As previously mentioned, it partitions tasks and has them run simultaneously to make the execution faster. They both pretty much share the same characteristics except from the fact that processes occupy their own memory space and threads share memory with other threads within the same process. This makes processes a bit more reliable because they can't corrupt other processes, but since threads share memory space, they can corrupt other threads within the same process. Just like a process, a thread has a stack to store local variables, function parameters and function calls, registers for fast location,

instructions, but as previously mentioned, shared memory space. That doesn't mean threads shouldn't be used. They are quite popular and if implemented correctly, quite efficient.

Like a process, threads have what is called a Thread Control Block (TCB) that performs very similar to a Process Control Block (PCB) in that it holds information specific to that thread's execution (thread ID, Program Counter, Register state and stack pointer). Like a process, a Thread has various states it can occupy, known as Thread states. A thread state lets us know if the thread is new, running on CPU or waiting for something to continue or finish execution.

Inside the thread world, there is also what is known as multi-threading, which is basically multiple threads running at the same time. As previously mentioned, threads have life cycles (new, runnable, running, blocked and terminated) that determine where the thread is currently situated. Multi-threading puts more load on the processor but is the best performance optimization tool as it improves application speed and responsiveness by running instructions in parallel. It behaves exactly like a supermarket that has multiple open registers scanning customer items, packing and dispatching. This performance reduces running time exponentially as it performs the same tasks simultaneously and gets them out of the way faster.

All major operating systems implement threads and multi-threading very similarly. Specifically talking about Windows, MacOS and Linux, they all support threading and multi-threading. Windows heavily relies on the User-Mode Scheduling that allows applications to schedule their own threads, making it easier for users to use threads on their Operating System. MacOS threads are usually at kernel-level, but thru the Gran Central Dispatch, users can manage them at user-level in some way. Linux, however, handles thread a little bit differently, as it basically treats them as lightweight processes. Linux handles processes and threads using the

same kernel structure called `task_struct`. The only difference lies in the fact that processes don't share memory space, and threads do (just like with Windows and MacOS).

In summary, a thread is a small portion of a process that can run simultaneously with other processes to speed up runtime. They share the same characteristics as processes (as they are basically smaller processes) but differentiate in the fact that they do share memory space with other threads and processes don't. Even though in definition, processes seem safer as they can't corrupt other processes, threads are not necessarily a bad idea as modern computers are expected to be faster and more efficient, and threads exist for that specific reason.

References:

- GeeksforGeeks. (2025, February 11). *Thread in operating system*. GeeksforGeeks. <https://www.geeksforgeeks.org/operating-systems/thread-in-operating-system/>
- CrashCourse. (2017, August 23). *Processes and threads: Crash course computer science #27* [Video]. YouTube. <https://www.youtube.com/watch?v=1myWEH8IGt4>
- Upwork. (2022, August 10). *What is multithreading? A beginner's guide*. Upwork. <https://www.upwork.com/resources/what-is-multithreading>
- Microsoft. (2025, July 14). *Processes and threads*. Microsoft Learn. <https://learn.microsoft.com/en-us/windows/win32/procthread/about-processes-and-threads>
- TutorialsPoint. (n.d.). *Threads vs processes in Linux*. TutorialsPoint. <https://www.tutorialspoint.com/article/threads-vs-processes-in-linux>